

ENSF 380

TOPIC 14: TESTING PART II

Logging as an Aid to Debugging

Track events and debug unexpected issues

Available without recompiling the code

Provide context which can help you understand the code later

Never forget to delete an errant print statement

Don't add the same print statements repeatedly

Levels of Severity in Logging

1. trace
2. debug
3. info
4. warn
5. error
6. fatal

Exercise 14.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Example Logging

01_Logging

```
private static int verbosity = 0;
private static Pattern myPattern = Pattern.compile("v",
    Pattern.CASE_INSENSITIVE);

if (args.length > 0) {
    int matchCount = 0;

    for(int i=0; i < args.length; i++) {
        Matcher myMatch = myPattern.matcher(args[i]);
        while (myMatch.find()) {
            matchCount++;
        }
    }
    verbosity = matchCount;
}
```

Example Logging

01_Logging

```
private static String info = "[INFO] %s\n";
private static String warn = "[WARNING] %s\n";

...

int sum = first + second;
if (this.verbosity >= 2) {
    System.out.printf(warn, "sum = "+sum);
}

...

if (this.verbosity >= 1) {
    System.out.printf(info, "return sum minus product");
}
```

Standardized solutions

- Add in extra context by default
- Handles file writing, log rotation
- Allows separation into different files/outputs based on severity
- Works well with other components

Example tools

- `org.apache.logging.log4j`
 - ...but be careful what version you use (avoid 2.0-beta7 – 2.17.0)
- Logback
- SLF4J
- JCL

Including External Libraries

Listing multiple paths

- Mac and Linux – use **:**
 - `<LibraryPath1>:<LibraryPath2>`
- Windows – use **;**
 - `<LibraryPath1>;<LibraryPath2>`
- Example (Mac)
 - `/Users/ann/Library/Java/Extensions/hamcrest-core-1.3.jar:/Users/ann/Library/Java/Extensions/junit-4.13.2.jar`
 - `/Users/ann/Library/Java/Extensions/*`

Important: Although we have explained how to use external libraries, you should still not include them in your code, except for the ones we explicitly tell you to use.

Important: Do not forget to include the current directory in the list of paths you provide with `-cp`

Instruct Java to use other libraries

- `-cp <LibraryPaths>`
- Example (Mac)
 - `javac -cp ./Users/ann/Library/Java/Extensions/* MyCodeTest.java`
 - `java -cp ./Users/ann/Library/Java/Extensions/* org.junit.runner.JUnit4Core MyCodeTest`
- Use file directory path for your own libraries when compiling, but specify package name when running
 - `javac -cp ./Users/ann/Library/Java/Extensions/* edu.ucalgary.oop.MyCodeTest.java`
 - `java -cp ./Users/ann/Library/Java/Extensions/* org.junit.runner.JUnit4Core edu.ucalgary.oop.MyCodeTest`

Unit Testing

Allows testing of individual code components- “units”

Usually tests public methods and data members

Compares expected output with actual output

Inputs vary over boundary conditions

Test-driven development (TDD)

Several Java testing frameworks available

JUnit

Open source framework for writing and running tests

Supported by many Java IDEs

Build automation tools available

Download and run JUnit 4 from the command line

<https://junit.org/junit4/>

Important: Regardless of your IDE, you must be able to compile and run tests from the command line.

Tip: Remember to replace XX in the tutorial commands with the correct version of JUnit 4.

Demo

INSTALL JUNIT

Exercise 14.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Test-Driven Development (TDD)

Test-driven development is a software development method that involves writing tests before writing code

The process is:

- Write a test which will fail
- Write code to pass the test
- Refactor the code to improve structure, readability, performance and maintainability
- Repeat the process for each new feature or modification

In this course

- You may be given tests and asked to write code against the tests
- You may be given tests and buggy code, and asked to fix the code using the tests
- You may be asked to write tests, then code
- You may have the option to practice TDD

TDD is a good development practice, but we sometimes use alternatives due to course constraints

Writing Code after Tests

If you are asked to write code against tests:

- Comment out all but one test. Pick a simple test, for instance of a constructor and getter
- Write the code to pass the test
- Uncomment the next test and repeat the process
- You may opt to handle all simple tests (constructors, getters, setters) first, or you may opt to complete the full functionality of a method (e.g., add error handling to a constructor) before moving to the next method

Debugging Code with Tests

If you are asked to debug code against tests:

- Apply the RIMGENT process
- Try to compile the code and fix any syntax errors which are preventing compilation
 - If there are multiple classes, try compiling the classes one at a time, starting with the class which has the least dependencies
- Try to compile the code against the tests and fix any syntax errors which are preventing compilation with the tests
 - Use the technique of compiling one class at a time to isolate errors
- Identify a failing test
 - Read the test to understand what is being tested
 - Examine the code to determine why it is failing the test
- Repeat for each failing test
 - Sometimes, fixing the code for one test will fix multiple tests (e.g., if a getter is returning the wrong value, fixing the getter will fix all tests which depend upon it)

Exercise 14.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Writing Tests, then Code

List the scenarios for the new feature

- List the expected variants of the new behavior: basic case and what-if (edge cases and boundary conditions)

Write a test for an item on the list

- Write a test which would pass if the code met the requirement for the variant

Repeat until all features have been considered

When you start to write the code, comment out other tests and focus on one at a time

Run all tests

- The new test should fail, other tests should pass

Write the simplest code that passes the test

Run all tests

- All tests should pass, not just the new one

Repeat until all tests pass

TDD

List the scenarios for the new feature

- List the expected variants of the new behavior: basic case and what-if (edge cases and boundary conditions)

Write a test for an item on the list

- Write a test which would pass if the code met the requirement for the variant

Run all tests

- The new test should fail, other tests should pass

Write the simplest code that passes the test

Run all tests

- All tests should pass, not just the new one

Repeat until all features have been implemented

Advantages of Writing Tests First

Improved code quality

Enhanced design

Better collaboration

How Many Tests is "Enough"?

Good code coverage is generally 80-90%

Code coverage is measured by determining how much of the program's source code is executed during testing

- Calculated as $\text{LoCExecutedByTests} / \text{TotalLoC}$ (Lines of Code)
- This determines which execution paths are covered by at least one test case

Test coverage vs Code coverage

- Test coverage is a more subjective measure, which involves thinking about "what if" scenarios

Code vs Test Coverage

Method

```
public static String  
appendConditionally(int number) {  
    StringBuilder result = new  
        StringBuilder("Start");  
  
    if (number > 10) {  
        result.append(" - Greater than 10");  
    }  
  
    result.append(" - Always appended")  
    return result.toString();  
}
```

Tests

appendConditionally(100)

Code coverage is 100%

Test coverage is insufficient

Code vs Test Coverage

Method

```
public static String categorizeAge(int age) {  
    if (age < 13) {  
        return "child";  
    } else if (age < 18) {  
        return "teenager";  
    } else if (age < 65) {  
        return "adult";  
    } else {  
        return "senior";  
    }  
}
```

Tests

```
categorizeAge(12);  
categorizeAge(14);  
categorizeAge(30);  
categorizeAge(90);
```

Code coverage is 100%

Test coverage is insufficient

- Boundary conditions (e.g., age = 13) and
- Edge cases (e.g., age = -1) are not tested!

Summary: Knowledge

Importance of unit testing

JUnit testing framework

Test execution

How logging can aid in debugging

How to use tests to aid in debugging and development

Code coverage vs test coverage

Summary: Skills

Including external libraries in Java compilation and execution from the command line

Setting up JUnit 4 from the command line

Executing simple tests from the command line

Debugging code using tests

Log with different levels of verbosity

Additional Information

Related reading

- Volume 1, Chapter 4.8: JAR Files